

Research Report

AI Workspace Platform

1. Introduction

Enterprise AI platforms such as ChatGPT Teams, Claude Projects, and Microsoft Copilot Studio have converged on a common pattern: isolated workspaces, configurable assistants, persistent memory, and grounded document access. This report examines that architecture pattern and how it was applied in building the AI Workspace Platform, a multi-user system supporting several LLM providers behind a single interface.

2. Comparable AI Platforms

Commercial AI workspace products share three structural traits: workspace-level isolation of data, assistant-level customization (system prompt, model, tone), and provider abstraction so the underlying model can change without disrupting the user experience. Claude Projects emphasizes long-context document grounding; ChatGPT Teams emphasizes shared custom GPTs across an organization; both rely on retrieval-augmented generation for document-grounded answers rather than fine-tuning per customer.

3. Workspace Architecture and Multi-Tenancy

A workspace acts as a tenancy boundary: every resource (conversation, document, memory item, prompt) is scoped to exactly one workspace, and every workspace is owned by exactly one user. This project implements that boundary at the API layer — every route checks workspace ownership before returning data — rather than relying solely on database-level row security, which keeps the authorization logic auditable in one place and is an appropriate trade-off for a single-tenant-per-request SQLite deployment.

4. Persistent Memory Strategies

Two dominant strategies exist for giving an assistant memory across sessions: automatic summarization of prior conversations injected into future prompts, and explicit user-curated memory items. Automatic summarization scales better but risks drifting or hallucinating what was 'remembered.' This project uses the explicit-curation model — memory items with categories and a pinned flag — trading some convenience for transparency and user control, which is preferable in an academic/evaluation context where memory recall accuracy needs to be measurable (see the accompanying Evaluation Framework).

5. Prompt Management and Reusability

Prompt libraries reduce repeated effort and encode organizational best practice into reusable templates. Categorizing templates (writing, programming, research, business, education, custom)

mirrors how commercial platforms organize prompt marketplaces, and keeping templates workspace-scoped avoids leaking one team's prompt content into another's workspace.

6. Knowledge Management via Retrieval-Augmented Generation

Grounding LLM answers in uploaded documents avoids two failure modes: hallucinated facts and stale training data. The standard RAG pipeline — extract, chunk, embed, retrieve by similarity, and inject the retrieved context into the prompt — was implemented with a lightweight offline embedding model (a hashing vectorizer) rather than a paid embeddings API, trading some retrieval quality for zero external dependency and zero marginal cost, which suits a student project's API budget while keeping the embedding interface identical to what a production embeddings API would expose.

7. Multi-User Systems and Data Isolation

Correct multi-tenant isolation requires that no user can read or modify another user's data through any code path, not just the 'obvious' ones. This was validated directly: the automated test suite includes a dedicated isolation test asserting that a second registered user sees zero workspaces belonging to the first, and that direct access to another user's workspace ID returns 404 rather than 403 (avoiding confirming the resource's existence to an unauthorized caller).

8. Scalability Considerations

The current SQLite-backed, single-process deployment is appropriate for coursework and small teams. The architecture is intentionally compatible with horizontal scaling: SQLAlchemy's engine can point at PostgreSQL by changing only `DATABASE_URL`, the vector storage layer is already abstracted behind a swappable embeddings module, and rate limiting can move from in-process memory to a shared Redis store without touching call sites, since `slowapi` supports a Redis storage backend natively.

9. Security Posture

Security decisions made in this project — bcrypt password hashing, JWT expiry, per-route ownership checks, upload type allow-listing, and per-IP rate limiting on authentication endpoints — follow standard practice for API-first web applications. A full discussion of risks and mitigations is in the accompanying Security Review document.

10. Conclusion

The AI Workspace Platform demonstrates that the architectural patterns underlying commercial multi-user AI products — workspace isolation, provider abstraction, RAG-based grounding, and explicit memory curation — can be implemented at a scope appropriate for a university capstone project without sacrificing correctness or testability, as evidenced by full automated test coverage across every module described in this report.